## 0. Install and import

```
%pip install -q "tensorflow>=2.16" "keras>=3.3" "transformers>=4.45,<5" "datasets>=3.2,<4" "scikit-learn>=1.5" "ma
```

```
import os
os.environ["KERAS_BACKEND"] = "tensorflow"

import json
import math
import time
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import keras
from keras import layers
from scipy.special import erf

from datasets import load_dataset
from transformers import AutoTokenizer
from huggingface_hub import hf_hub_download
from safetensors.numpy import load_file
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score, f1_score, classification_report, confusion_matrix
)

SEED = 42
keras.utils.set_random_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)

print("TensorFlow:", tf.__version__)
print("Keras:", keras.__version__)
print("Devices:", tf.config.list_physical_devices())
```

```
TensorFlow: 2.21.0
Keras: 3.13.2
Devices: [PhysicalDevice(name='/physical_device:CPU:0', device_type='CPU'), PhysicalDevice(name='/physical_device
```

## 1. Configuration

The architecture numbers must match the DistilBERT checkpoint, so they are fixed. Change `DATASET` to switch between Banking77 and Bitext.

```
class ModelConfig:
    # Pretrained model on Hugging Face
```

```
    PRETRAINED_NAME = "distilbert/distilbert-base-uncased"
    TOKENIZER_NAME = "distilbert/distilbert-base-uncased"

    # DistilBERT architecture (must match the checkpoint)
    VOCAB_SIZE_EXPECTED = 30522
    CONTEXT_LENGTH = 512       # number of learned position embeddings
    NUM_LAYERS = 6
    NUM_HEADS = 12
    EMBED_DIM = 768
    FF_DIM = 3072
    DROPOUT = 0.1

    # Supervised task
    DATASET = "banking77"      # "banking77" or "bitext"
    MAX_LENGTH = 64            # tokens per message (short messages, so 64 is enough)
    SUPPORT_LIMIT = None       # only used by Bitext

    FINETUNE_EPOCHS = 8
    FINETUNE_BATCH_SIZE = 32
    FINETUNE_LR = 5e-5

    OUTPUT_DIR = "keras_hf_distilbert_project"

cfg = ModelConfig()
```

## Save outputs to Google Drive (recommended on free Colab)

The Colab disk is wiped when the session ends. This cell mounts Drive so the pretrained backbone and every fine-tuned model survive. Outside Colab it just uses a local folder.

```
try:
    from google.colab import drive
    drive.mount("/content/drive")
    cfg.OUTPUT_DIR = "/content/drive/MyDrive/keras_hf_distilbert_project"
except ImportError:
    pass

BASE_DIR = os.path.join(cfg.OUTPUT_DIR, "distilbert_base")   # clean pretrained backbone
RUN_DIR = os.path.join(cfg.OUTPUT_DIR, cfg.DATASET)           # one folder per dataset
os.makedirs(BASE_DIR, exist_ok=True)
os.makedirs(RUN_DIR, exist_ok=True)
print("Base model folder:", BASE_DIR)
print("This run folder:  ", RUN_DIR)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force
Base model folder: /content/drive/MyDrive/keras_hf_distilbert_project/distilbert_base
This run folder:   /content/drive/MyDrive/keras_hf_distilbert_project/banking77
```

## 2. DistilBERT tokenizer

DistilBERT uses a lowercase WordPiece tokenizer. It already has a `[PAD]` token (id 0), so nothing needs to be added. Every message becomes `[CLS] ... [SEP]`, and the `[CLS]` position is the one we classify from.

```
tokenizer = AutoTokenizer.from_pretrained(cfg.TOKENIZER_NAME)

PAD_ID = tokenizer.pad_token_id
VOCAB_SIZE = len(tokenizer)
assert VOCAB_SIZE == cfg.VOCAB_SIZE_EXPECTED, f"Unexpected vocabulary size: {VOCAB_SIZE}"

print("Vocabulary:", VOCAB_SIZE)
print("PAD:", tokenizer.pad_token, PAD_ID)
print("CLS:", tokenizer.cls_token, tokenizer.cls_token_id)
print("SEP:", tokenizer.sep_token, tokenizer.sep_token_id)

sample_text = "I need help tracking my order."
sample_ids = tokenizer(sample_text)["input_ids"]
print(sample_ids)
print(tokenizer.convert_ids_to_tokens(sample_ids))
```

```
/usr/local/lib/python3.13/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/t
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 3.09kB/s]
config.json: 100%  483/483 [00:00<00:00, 25.0kB/s]
vocab.txt:  232k/? [00:00<00:00, 9.83MB/s]
tokenizer.json:  466k/? [00:00<00:00, 22.9MB/s]
Vocabulary: 30522
PAD: [PAD] 0
CLS: [CLS] 101
SEP: [SEP] 102
[101, 1045, 2342, 2393, 9651, 2026, 2344, 1012, 102]
['[CLS]', 'i', 'need', 'help', 'tracking', 'my', 'order', '.', '[SEP]']
```

## 3. Build the DistilBERT architecture using Keras subclassing

We create three classes:

1. `EncoderBlock(keras.layers.Layer)`: bidirectional multi-head self-attention + feed-forward network, with the LayerNorm **after** each part.
2. `DistilBertBackbone(keras.Model)`: token embeddings, positional embeddings, embedding LayerNorm, and stacked encoder blocks.
3. `DistilBertIntentClassifier(keras.Model)`: backbone + classification head (built in section 9).

The Keras subclass pattern is:

```
class MyModel(keras.Model):
    def __init__(self, ...):
        super().__init__()
        # Define layers here

    def call(self, inputs, training=False):
        # Define forward computation here
        return outputs
```

`call()` is the forward pass. Calling `model(inputs)` invokes it.

```
class EncoderBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, dropout=0.1, **kwargs):
        super().__init__(**kwargs)

        self.attention = layers.MultiHeadAttention(
            num_heads=num_heads,
            key_dim=embed_dim // num_heads,
            dropout=dropout
        )

        self.attention_dropout = layers.Dropout(dropout)
        self.norm1 = layers.LayerNormalization(epsilon=1e-12)   # sa_layer_norm

        self.ffn = keras.Sequential([
            layers.Dense(ff_dim, activation="gelu"),
            layers.Dense(embed_dim)
        ])

        self.ffn_dropout = layers.Dropout(dropout)
        self.norm2 = layers.LayerNormalization(epsilon=1e-12)   # output_layer_norm

    def call(self, inputs, attention_mask=None, training=False):
        # attention_mask shape: [batch, 1, sequence] (padding mask only, no causal mask)
        attention_output = self.attention(
            query=inputs,
            value=inputs,
            key=inputs,
            attention_mask=attention_mask,
            training=training
        )

        # Post-normalization: add first, then LayerNorm
        x = self.norm1(inputs + self.attention_dropout(attention_output, training=training))
```

```
        ffn_output = self.ffn(x, training=training)
        return self.norm2(x + self.ffn_dropout(ffn_output, training=training))
```

## DistilBERT backbone

Position *t* can look at **every** other position, left and right. The only thing hidden from attention is padding.

```
class DistilBertBackbone(keras.Model):
    def __init__(
        self,
        vocab_size,
        max_length,
        embed_dim,
        num_heads,
        num_layers,
        ff_dim,
        dropout=0.1,
        **kwargs
    ):
        super().__init__(**kwargs)

        self.max_length = max_length
        self.token_embedding = layers.Embedding(vocab_size, embed_dim)
        self.position_embedding = layers.Embedding(max_length, embed_dim)
        self.embedding_norm = layers.LayerNormalization(epsilon=1e-12)
        self.embedding_dropout = layers.Dropout(dropout)

        self.encoder_blocks = [
            EncoderBlock(
                embed_dim=embed_dim,
                num_heads=num_heads,
                ff_dim=ff_dim,
                dropout=dropout,
                name=f"encoder_block_{i+1}"
            )
            for i in range(num_layers)
        ]

    def call(self, inputs, training=False):
        token_ids = inputs["token_ids"]
        attention_mask = inputs.get("attention_mask", None)

        sequence_length = tf.shape(token_ids)[1]
        positions = tf.range(start=0, limit=sequence_length, delta=1)

        x = self.token_embedding(token_ids) + self.position_embedding(positions)
        x = self.embedding_norm(x)
        x = self.embedding_dropout(x, training=training)

        # Convert [batch, sequence] key mask to [batch, 1, sequence].
        key_mask = None
        if attention_mask is not None:
            key_mask = tf.cast(attention_mask[:, tf.newaxis, :], tf.bool)

        for block in self.encoder_blocks:
            x = block(x, attention_mask=key_mask, training=training)

        return x
```

# 4. Build the model and load the pretrained DistilBERT weights

First we create the backbone with random weights using the DistilBERT sizes. Then the conversion function copies every pretrained tensor into it:

- Hugging Face stores `nn.Linear` weights as `[out, in]`, but a Keras `Dense` kernel is `[in, out]`, so every linear weight is **transposed**.
- The query, key and value matrices are reshaped per attention head, and so is the output projection.
- `embeddings.LayerNorm` goes to `embedding_norm`, `sa_layer_norm` to `norm1`, and `output_layer_norm` to `norm2`.
- The masked-language-model head (`vocab_transform`, `vocab_layer_norm`, `vocab_projector`) is **not** part of the backbone. We keep those tensors aside only for the sanity check in section 5.

```
def build_backbone():
    return DistilBertBackbone(
        vocab_size=VOCAB_SIZE,
```

```
        max_length=cfg.CONTEXT_LENGTH,
        embed_dim=cfg.EMBED_DIM,
        num_heads=cfg.NUM_HEADS,
        num_layers=cfg.NUM_LAYERS,
        ff_dim=cfg.FF_DIM,
        dropout=cfg.DROPOUT,
        name="distilbert_backbone"
    )

backbone = build_backbone()

# Build all layers before copying weights.
dummy = {
    "token_ids": tf.zeros((2, cfg.MAX_LENGTH), dtype=tf.int32),
    "attention_mask": tf.ones((2, cfg.MAX_LENGTH), dtype=tf.int32)
}
_ = backbone(dummy)

print("Backbone parameters:", f"{backbone.count_params():,}")
```

```
Backbone parameters: 66,362,880
```

```
def convert_hf_distilbert_weights(backbone, tensors, num_layers, num_heads, embed_dim):
    """Copy DistilBERT tensors (Hugging Face layout) into the Keras DistilBertBackbone.

    Hugging Face stores nn.Linear weights as [out, in], so they are transposed to the
    Keras Dense layout [in, out]. Query / key / value / output matrices are then
    reshaped per attention head.
    """
    t = {k.replace("distilbert.", "", 1): v for k, v in tensors.items()}
    head_dim = embed_dim // num_heads

    def get(name):
        if name not in t:
            raise KeyError(f"{name} not found. Available keys: {list(t)[:12]} ...")
        return np.asarray(t[name], dtype="float32")

    def linear(name):
        return get(name + ".weight").T, get(name + ".bias")   # transpose [out, in] -> [in, out]

    backbone.token_embedding.set_weights([get("embeddings.word_embeddings.weight")])
    backbone.position_embedding.set_weights([get("embeddings.position_embeddings.weight")])
    backbone.embedding_norm.set_weights([get("embeddings.LayerNorm.weight"), get("embeddings.LayerNorm.bias")])

    for i, block in enumerate(backbone.encoder_blocks[:num_layers]):
        p = f"transformer.layer.{i}."

        for dense, name in [
            (block.attention.query_dense, "attention.q_lin"),
            (block.attention.key_dense, "attention.k_lin"),
            (block.attention.value_dense, "attention.v_lin"),
        ]:
            w, b = linear(p + name)
            dense.set_weights([w.reshape(embed_dim, num_heads, head_dim), b.reshape(num_heads, head_dim)])

        w, b = linear(p + "attention.out_lin")
        block.attention.output_dense.set_weights([w.reshape(num_heads, head_dim, embed_dim), b])

        block.norm1.set_weights([get(p + "sa_layer_norm.weight"), get(p + "sa_layer_norm.bias")])

        block.ffn.layers[0].set_weights(list(linear(p + "ffn.lin1")))
        block.ffn.layers[1].set_weights(list(linear(p + "ffn.lin2")))

        block.norm2.set_weights([get(p + "output_layer_norm.weight"), get(p + "output_layer_norm.bias")])
```

```
weights_path = hf_hub_download(repo_id=cfg.PRETRAINED_NAME, filename="model.safetensors")
hf_tensors = load_file(weights_path)
print(len(hf_tensors), "tensors in the checkpoint. Example keys:", list(hf_tensors)[:4])

convert_hf_distilbert_weights(
    backbone, hf_tensors,
    num_layers=cfg.NUM_LAYERS, num_heads=cfg.NUM_HEADS, embed_dim=cfg.EMBED_DIM
)
print("Pretrained DistilBERT weights loaded into the Keras backbone.")
```

```
model.safetensors:  100%                                268M/268M  [00:02<00:00,  149MB/s]
105 tensors in the checkpoint. Example keys: ['distilbert.embeddings.LayerNorm.bias', 'distilbert.embeddings.Laye
Pretrained DistilBERT weights loaded into the Keras backbone.
```

## Look at the loaded weights

Every weight tensor in the backbone with its shape. Embedding block first, then the first encoder block (the other five repeat the same pattern).

```
rows = []
for w in backbone.weights:
    rows.append({"weight": w.path, "shape": tuple(w.shape), "parameters": int(np.prod(w.shape))})
weights_df = pd.DataFrame(rows)

print("Total weight tensors:", len(weights_df))
print("Total parameters:", f"{weights_df['parameters'].sum():,}")
weights_df[~weights_df["weight"].str.contains("encoder_block_[2-6]")]
```

```
Total weight tensors: 100
Total parameters: 66,362,880
```

| | weight | shape | parameters |
|---|---|---|---|
| 0 | distilbert_backbone/embedding/embeddings | (30522, 768) | 23440896 |
| 1 | distilbert_backbone/embedding_1/embeddings | (512, 768) | 393216 |
| 2 | distilbert_backbone/layer_normalization/gamma | (768,) | 768 |
| 3 | distilbert_backbone/layer_normalization/beta | (768,) | 768 |
| 4 | distilbert_backbone/encoder_block_1/multi_head... | (768, 12, 64) | 589824 |
| 5 | distilbert_backbone/encoder_block_1/multi_head... | (12, 64) | 768 |
| 6 | distilbert_backbone/encoder_block_1/multi_head... | (768, 12, 64) | 589824 |
| 7 | distilbert_backbone/encoder_block_1/multi_head... | (12, 64) | 768 |
| 8 | distilbert_backbone/encoder_block_1/multi_head... | (768, 12, 64) | 589824 |
| 9 | distilbert_backbone/encoder_block_1/multi_head... | (12, 64) | 768 |
| 10 | distilbert_backbone/encoder_block_1/multi_head... | (12, 64, 768) | 589824 |
| 11 | distilbert_backbone/encoder_block_1/multi_head... | (768,) | 768 |
| 12 | distilbert_backbone/encoder_block_1/layer_norm... | (768,) | 768 |
| 13 | distilbert_backbone/encoder_block_1/layer_norm... | (768,) | 768 |
| 14 | distilbert_backbone/encoder_block_1/sequential... | (768, 3072) | 2359296 |
| 15 | distilbert_backbone/encoder_block_1/sequential... | (3072,) | 3072 |
| 16 | distilbert_backbone/encoder_block_1/sequential... | (3072, 768) | 2359296 |
| 17 | distilbert_backbone/encoder_block_1/sequential... | (768,) | 768 |
| 18 | distilbert_backbone/encoder_block_1/layer_norm... | (768,) | 768 |
| 19 | distilbert_backbone/encoder_block_1/layer_norm... | (768,) | 768 |

## 5. Sanity check: did the pretrained weights load correctly?

A wrongly loaded or random backbone cannot fill in a missing word. DistilBERT was pretrained to predict a hidden `[MASK]` token, so we run its masked-language-model head on top of our Keras backbone (with plain NumPy) and see if it guesses the missing word.

```
# Masked-language-model head tensors (only used for this check).
mlm_w = hf_tensors["vocab_transform.weight"].T
mlm_b = hf_tensors["vocab_transform.bias"]
mlm_norm_w = hf_tensors["vocab_layer_norm.weight"]
mlm_norm_b = hf_tensors["vocab_layer_norm.bias"]
mlm_out_b = hf_tensors["vocab_projector.bias"]
word_embeddings = hf_tensors["distilbert.embeddings.word_embeddings.weight"]   # output layer is tied to it

def fill_mask(text, top_k=5):
    encoded = tokenizer(text, return_tensors="np")
    hidden = backbone(
        {"token_ids": encoded["input_ids"], "attention_mask": encoded["attention_mask"]},
        training=False
    ).numpy()

    mask_position = int(np.where(encoded["input_ids"][0] == tokenizer.mask_token_id)[0][0])
    h = hidden[0, mask_position] @ mlm_w + mlm_b
    h = 0.5 * h * (1.0 + erf(h / math.sqrt(2.0)))                    # exact GELU
    h = (h - h.mean()) / np.sqrt(h.var() + 1e-12) * mlm_norm_w + mlm_norm_b
    logits = h @ word_embeddings.T + mlm_out_b
```

```
    best = np.argsort(logits)[::-1][:top_k]
    return tokenizer.convert_ids_to_tokens(best.tolist())

print(fill_mask("The capital of France is [MASK]."))
print(fill_mask("I forgot my [MASK] and cannot log in to my bank account."))
```

```
['marseille', 'nantes', 'toulouse', 'paris', 'lyon']
['wallet', 'password', 'phone', 'passport', 'savings']
```

## 6. Save the pretrained backbone (reusable base model)

We save the clean pretrained backbone **before any fine-tuning**: the weights, a `config.json` with the architecture sizes, and the tokenizer. `load_backbone()` rebuilds the model from that folder, so every new dataset starts from the same clean copy, even in a new session.

```
BASE_CONFIG = {
    "vocab_size": VOCAB_SIZE,
    "max_length": cfg.CONTEXT_LENGTH,
    "embed_dim": cfg.EMBED_DIM,
    "num_heads": cfg.NUM_HEADS,
    "num_layers": cfg.NUM_LAYERS,
    "ff_dim": cfg.FF_DIM,
    "dropout": cfg.DROPOUT,
}

def save_backbone(backbone, config, tokenizer, path):
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, "config.json"), "w") as f:
        json.dump(config, f, indent=2)
    backbone.save_weights(os.path.join(path, "backbone.weights.h5"))
    tokenizer.save_pretrained(os.path.join(path, "tokenizer"))

def load_backbone(path):
    with open(os.path.join(path, "config.json")) as f:
        config = json.load(f)
    new_backbone = DistilBertBackbone(**config, name="distilbert_backbone")
    _ = new_backbone({
        "token_ids": tf.zeros((1, 8), dtype=tf.int32),
        "attention_mask": tf.ones((1, 8), dtype=tf.int32)
    })
    new_backbone.load_weights(os.path.join(path, "backbone.weights.h5"))
    return new_backbone

save_backbone(backbone, BASE_CONFIG, tokenizer, BASE_DIR)
print("Saved pretrained backbone to:", BASE_DIR)
```

```
Saved pretrained backbone to: /content/drive/MyDrive/keras_hf_distilbert_project/distilbert_base
```

## 7. Load the supervised dataset

Set `cfg.DATASET` in the configuration:

- **`banking77`** (`PolyAI/banking77`): 77 fine-grained banking intents. It has a train and a test split, so 10% of train becomes the validation set.
- **`bitext`**: the customer-support dataset from the scratch notebook, split 80/10/10.

If `load_dataset` fails for Banking77, the cell automatically falls back to the original CSV files from the PolyAI GitHub repository.

```
BANKING77_TRAIN_URL = "https://raw.githubusercontent.com/PolyAI-LDN/task-specific-datasets/master/banking_data/t
BANKING77_TEST_URL = "https://raw.githubusercontent.com/PolyAI-LDN/task-specific-datasets/master/banking_data/te

def load_banking77_frames():
    try:
        ds = load_dataset("PolyAI/banking77")
        names = ds["train"].features["label"].names
        frames = []
        for split in ("train", "test"):
            df = ds[split].to_pandas()[["text", "label"]].dropna()
            df["intent"] = df["label"].map(lambda i: names[int(i)])
            frames.append(df[["text", "intent"]])
        return frames[0], frames[1]
    except Exception as error:
        print("load_dataset failed:", repr(error)[:200])
        print("Falling back to the original CSV files on GitHub.")
        train_df = pd.read_csv(BANKING77_TRAIN_URL).rename(columns={"category": "intent"})
        test_df = pd.read_csv(BANKING77_TEST_URL).rename(columns={"category": "intent"})
```

```
        return train_df[["text", "intent"]], test_df[["text", "intent"]]

if cfg.DATASET == "banking77":
    train_df, test_df = load_banking77_frames()
    for df in (train_df, test_df):
        df["text"] = df["text"].astype(str).str.strip()
        df["intent"] = df["intent"].astype(str).str.strip()

    label_encoder = LabelEncoder()
    label_encoder.fit(pd.concat([train_df["intent"], test_df["intent"]]))

    class_names = list(label_encoder.classes_)
    num_classes = len(class_names)

    X_full = train_df["text"].to_numpy(dtype=str)
    y_full = label_encoder.transform(train_df["intent"]).astype(np.int32)

    X_train, X_valid, y_train, y_valid = train_test_split(
        X_full, y_full, test_size=0.10, random_state=SEED, stratify=y_full
    )
    X_test = test_df["text"].to_numpy(dtype=str)
    y_test = label_encoder.transform(test_df["intent"]).astype(np.int32)

elif cfg.DATASET == "bitext":
    support = load_dataset(
        "bitext/Bitext-customer-support-llm-chatbot-training-dataset"
    )
    support_df = support["train"].to_pandas()[["instruction", "intent"]].dropna()

    support_df["instruction"] = support_df["instruction"].astype(str).str.strip()
    support_df["intent"] = support_df["intent"].astype(str).str.strip()
    support_df = support_df[
        (support_df["instruction"].str.len() > 0) &
        (support_df["intent"].str.len() > 0)
    ].reset_index(drop=True)

    if cfg.SUPPORT_LIMIT is not None:
        support_df = support_df.groupby("intent", group_keys=False).apply(
            lambda group: group.sample(
                n=min(len(group), max(1, cfg.SUPPORT_LIMIT // support_df["intent"].nunique())),
                random_state=SEED
            )
        ).reset_index(drop=True)

    label_encoder = LabelEncoder()
    support_df["label"] = label_encoder.fit_transform(support_df["intent"])

    class_names = list(label_encoder.classes_)
    num_classes = len(class_names)

    X = support_df["instruction"].to_numpy(dtype=str)
    y = support_df["label"].to_numpy(dtype=np.int32)

    X_train, X_temp, y_train, y_temp = train_test_split(
        X, y, test_size=0.20, random_state=SEED, stratify=y
    )
    X_valid, X_test, y_valid, y_test = train_test_split(
        X_temp, y_temp, test_size=0.50, random_state=SEED, stratify=y_temp
    )

    print("Train:", len(X_train), "Validation:", len(X_valid), "Test:", len(X_test))
    print("Number of classes:", num_classes)
    print(class_names)

else:
    raise ValueError("cfg.DATASET must be 'banking77' or 'bitext'")

print("Dataset:", cfg.DATASET)
print("Train:", len(X_train), "Validation:", len(X_valid), "Test:", len(X_test))
print("Number of classes:", num_classes)
print(class_names[:10], "...")
```

```
README.md:          13.0k/?  [00:00<00:00,  211kB/s]
data/train-00000-of-00001.parquet:  100%                                294k/294k  [00:00<00:00,  1.77MB/s]
data/test-00000-of-00001.parquet:  100%                                 92.4k/92.4k  [00:00<00:00,  643kB/s]
Generating  train  split:  100%                                 10003/10003  [00:00<00:00,  83791.26  examples/s]
Generating  test  split:  100%                                 3080/3080  [00:00<00:00,  91848.90  examples/s]
Dataset: banking77
Train: 9002 Validation: 1001 Test: 3080
Number of classes: 77
['Refund_not_showing_up', 'activate_my_card', 'age_limit', 'apple_pay_or_google_pay', 'atm_support', 'automatic_t
```

## 8. Tokenize the messages

The same DistilBERT tokenizer is used. This is essential because the pretrained token embedding matrix is indexed by the tokenizer's token IDs.

```
def tokenize_classification_texts(texts, max_length):
    encoded = tokenizer(
        list(texts),
        max_length=max_length,
        truncation=True,
        padding="max_length",
        return_attention_mask=True
    )
    return (
        np.asarray(encoded["input_ids"], dtype="int32"),
        np.asarray(encoded["attention_mask"], dtype="int32")
    )

train_ids, train_masks = tokenize_classification_texts(X_train, cfg.MAX_LENGTH)
valid_ids, valid_masks = tokenize_classification_texts(X_valid, cfg.MAX_LENGTH)
test_ids, test_masks = tokenize_classification_texts(X_test, cfg.MAX_LENGTH)

print(train_ids.shape, train_masks.shape)
```

```
(9002, 64) (9002, 64)
```

```
def make_classification_tf_dataset(
    token_ids, attention_masks, labels, batch_size, training=False
):
    ds = tf.data.Dataset.from_tensor_slices((
        {
            "token_ids": token_ids,
            "attention_mask": attention_masks
        },
        labels
    ))
    if training:
        ds = ds.shuffle(
            min(len(labels), 10000),
            seed=SEED,
            reshuffle_each_iteration=True
        )
    return ds.batch(batch_size).prefetch(tf.data.AUTOTUNE)

train_ds = make_classification_tf_dataset(
    train_ids, train_masks, y_train, cfg.FINETUNE_BATCH_SIZE, training=True
)
valid_ds = make_classification_tf_dataset(
    valid_ids, valid_masks, y_valid, cfg.FINETUNE_BATCH_SIZE
)
test_ds = make_classification_tf_dataset(
    test_ids, test_masks, y_test, cfg.FINETUNE_BATCH_SIZE
)
```

## 9. Build the subclassed DistilBERT intent classifier

We reuse the pretrained `DistilBertBackbone` and add a classification head.

Because the encoder is bidirectional, the `[CLS]` token (index 0) has already looked at the whole message, so its hidden state is the sentence vector. We pass it through dropout and a dense layer to produce intent logits.

```
class DistilBertIntentClassifier(keras.Model):
    def __init__(self, backbone, num_classes, dropout=0.2, **kwargs):
        super().__init__(**kwargs)
        self.backbone = backbone
        self.dropout = layers.Dropout(dropout)
        self.classifier = layers.Dense(num_classes, name="intent_logits")

    def call(self, inputs, training=False):
        hidden_states = self.backbone(
            {
                "token_ids": inputs["token_ids"],
                "attention_mask": inputs["attention_mask"]
            },
            training=training
        )

        cls_state = hidden_states[:, 0, :]    # [CLS] token
        x = self.dropout(cls_state, training=training)
        return self.classifier(x)
```

## 10. Transfer weights and fine-tune

We always create the classifier from a **fresh copy** of the saved pretrained backbone. Fine-tuning changes the backbone weights in place, so reusing the Python object from section 4 would make the second dataset start from an already fine-tuned model.

Training uses sparse categorical cross-entropy with early stopping on validation accuracy. The fit time is recorded for the comparison table.

```
fresh_backbone = load_backbone(BASE_DIR)

intent_model = DistilBertIntentClassifier(
    backbone=fresh_backbone,
    num_classes=num_classes,
    dropout=0.2,
    name="distilbert_intent_classifier"
)

_ = intent_model({
    "token_ids": tf.zeros((2, cfg.MAX_LENGTH), dtype=tf.int32),
    "attention_mask": tf.ones((2, cfg.MAX_LENGTH), dtype=tf.int32)
})

intent_model.summary()
```

**Model: "distilbert_intent_classifier"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| distilbert_backbone (DistilBertBackbone) | ? | 66,362,880 |
| dropout_38 (Dropout) | ? | 0 |
| intent_logits (Dense) | (2, 77) | 59,213 |

**Total params:** 66,422,093 (253.38 MB)
**Trainable params:** 66,422,093 (253.38 MB)
**Non-trainable params:** 0 (0.00 B)

```
intent_model.compile(
    optimizer=keras.optimizers.AdamW(
        learning_rate=cfg.FINETUNE_LR,
        weight_decay=0.01,
        clipnorm=1.0
    ),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="accuracy")]
)

finetune_callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=2,
        mode="max",
        restore_best_weights=True
    ),
    keras.callbacks.ModelCheckpoint(
        filepath=os.path.join(RUN_DIR, "best_intent_classifier.weights.h5"),
        monitor="val_accuracy",
        mode="max",
        save_best_only=True,
```

```
        save_weights_only=True
    )
]

finetune_start = time.time()
finetune_history = intent_model.fit(
    train_ds,
    validation_data=valid_ds,
    epochs=cfg.FINETUNE_EPOCHS,
    callbacks=finetune_callbacks
)
finetune_seconds = time.time() - finetune_start
print("Fine-tuning time (seconds):", round(finetune_seconds, 1))
```

```
Epoch 1/8
282/282 ━━━━━━━━━━━━━━━━━━━━ 196s 363ms/step - accuracy: 0.5881 - loss: 1.7317 - val_accuracy: 0.8761 - val_loss:
Epoch 2/8
282/282 ━━━━━━━━━━━━━━━━━━━━ 87s 309ms/step - accuracy: 0.9020 - loss: 0.3659 - val_accuracy: 0.8931 - val_loss:
Epoch 3/8
282/282 ━━━━━━━━━━━━━━━━━━━━ 73s 259ms/step - accuracy: 0.9472 - loss: 0.1893 - val_accuracy: 0.9131 - val_loss:
Epoch 4/8
282/282 ━━━━━━━━━━━━━━━━━━━━ 69s 246ms/step - accuracy: 0.9733 - loss: 0.1010 - val_accuracy: 0.9111 - val_loss:
Epoch 5/8
282/282 ━━━━━━━━━━━━━━━━━━━━ 82s 245ms/step - accuracy: 0.9818 - loss: 0.0663 - val_accuracy: 0.9111 - val_loss:
Fine-tuning time (seconds): 507.1
```

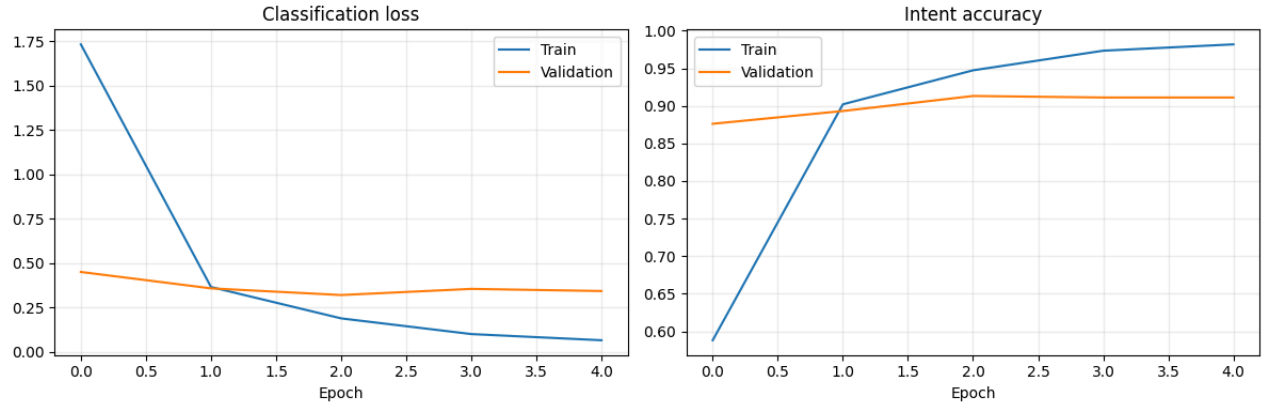
## 11. Plot fine-tuning results

```
history_df = pd.DataFrame(finetune_history.history)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history_df["loss"], label="Train")
axes[0].plot(history_df["val_loss"], label="Validation")
axes[0].set_title("Classification loss")
axes[0].set_xlabel("Epoch")
axes[0].legend()
axes[0].grid(alpha=0.25)

axes[1].plot(history_df["accuracy"], label="Train")
axes[1].plot(history_df["val_accuracy"], label="Validation")
axes[1].set_title("Intent accuracy")
axes[1].set_xlabel("Epoch")
axes[1].legend()
axes[1].grid(alpha=0.25)

plt.tight_layout()
plt.show()
```




## How far did fine-tuning move the weights?

We reload the clean pretrained backbone and compare it with the fine-tuned one, part by part. The number is the relative change: `norm(tuned - pretrained) / norm(pretrained)`. A small value means that part stayed close to the pretrained weights. The classification head is new, so it is not in this comparison.

```
base_backbone = load_backbone(BASE_DIR)
tuned_backbone = intent_model.backbone
```

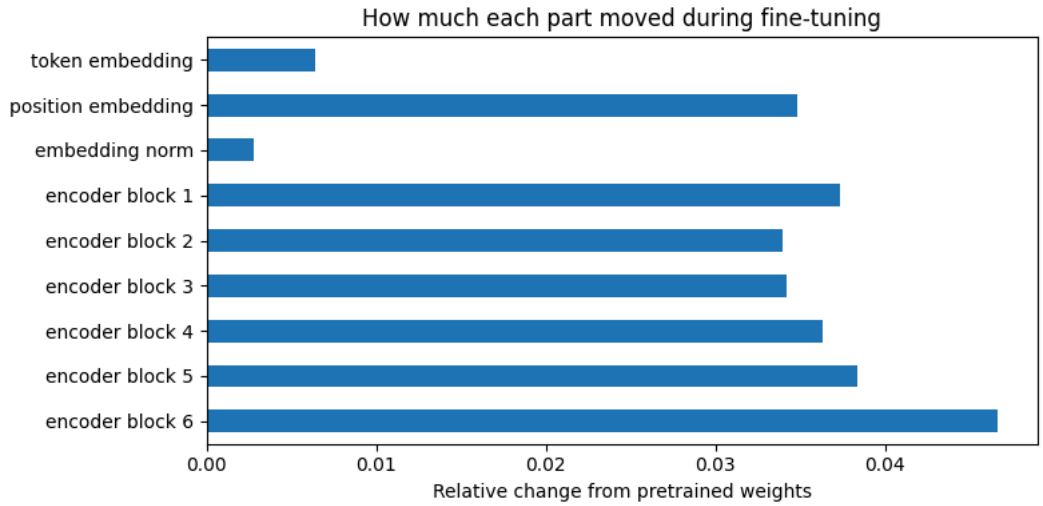
```
def relative_change(base_layer, tuned_layer):
    base = np.concatenate([w.numpy().ravel() for w in base_layer.weights])
    tuned = np.concatenate([w.numpy().ravel() for w in tuned_layer.weights])
    return float(np.linalg.norm(tuned - base) / np.linalg.norm(base))

parts = {
    "token embedding": (base_backbone.token_embedding, tuned_backbone.token_embedding),
    "position embedding": (base_backbone.position_embedding, tuned_backbone.position_embedding),
    "embedding norm": (base_backbone.embedding_norm, tuned_backbone.embedding_norm),
}
for i in range(cfg.NUM_LAYERS):
    parts[f"encoder block {i+1}"] = (base_backbone.encoder_blocks[i], tuned_backbone.encoder_blocks[i])

change_df = pd.DataFrame({
    "relative_change": {name: relative_change(a, b) for name, (a, b) in parts.items()}
})
print(change_df.round(5))

change_df.plot.barh(legend=False, figsize=(8, 4))
plt.gca().invert_yaxis()
plt.xlabel("Relative change from pretrained weights")
plt.title("How much each part moved during fine-tuning")
plt.tight_layout()
plt.show()
```

```
                    relative_change
token embedding             0.00639
position embedding          0.03482
embedding norm              0.00279
encoder block 1             0.03731
encoder block 2             0.03394
encoder block 3             0.03414
encoder block 4             0.03630
encoder block 5             0.03839
encoder block 6             0.04665
```




## 12. Evaluate on the held-out test set

```
test_loss, test_accuracy = intent_model.evaluate(test_ds, verbose=0)
print("Test loss:", round(float(test_loss), 4))
print("Test accuracy:", round(float(test_accuracy), 4))

all_logits = intent_model.predict(test_ds, verbose=0)
y_pred = np.argmax(all_logits, axis=1)

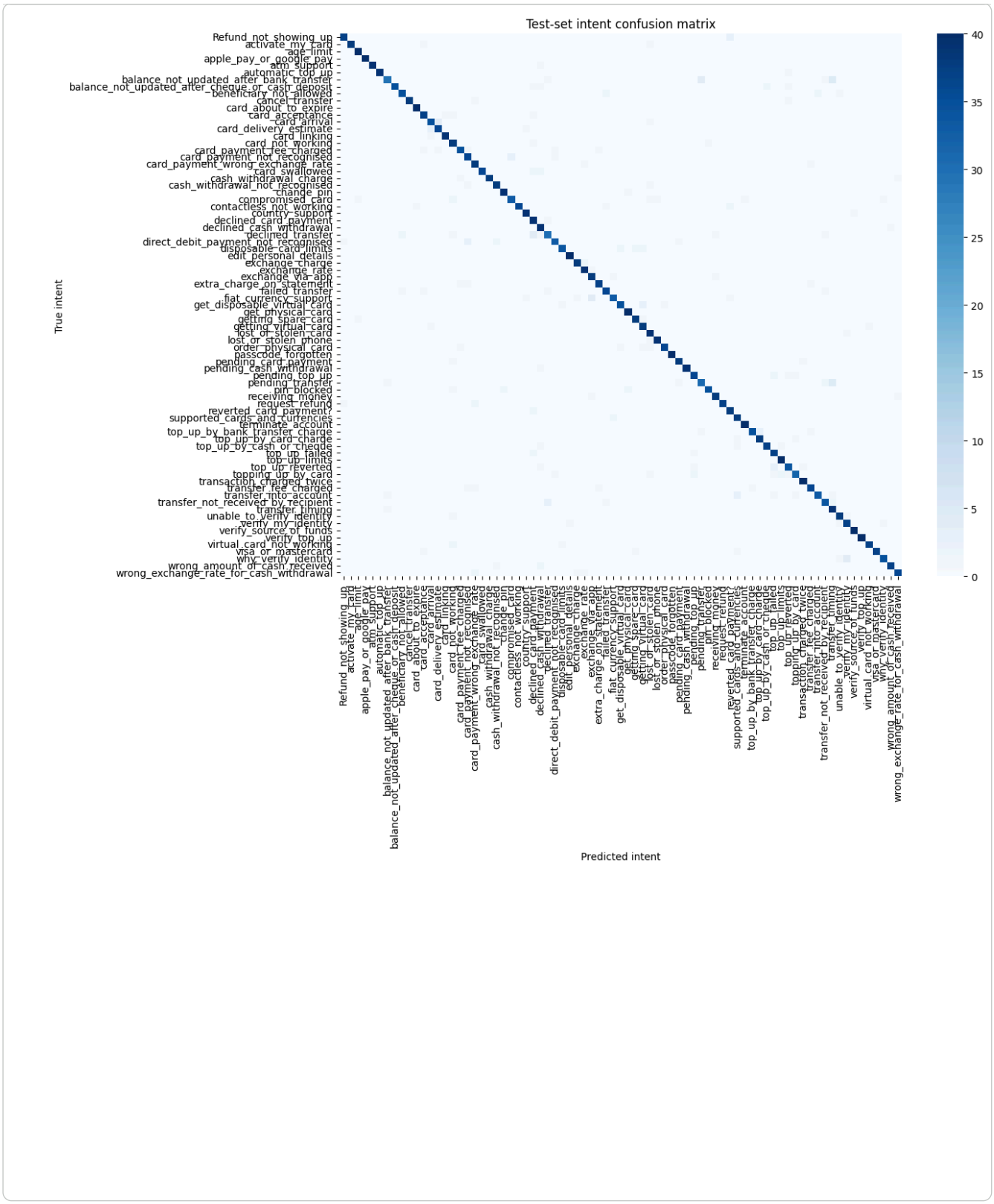
print(classification_report(
    y_test,
    y_pred,
    labels=np.arange(num_classes),
    target_names=class_names,
    zero_division=0
))
```

```
                          disposable_card_limits       0.94      0.85      0.89        40
                           edit_personal_details       0.98      1.00      0.99        40
                                 exchange_charge       0.95      0.95      0.95        40
                                   exchange_rate       0.91      0.97      0.94        40
                                exchange_via_app       0.86      0.95      0.90        40
                      extra_charge_on_statement       0.95      0.93      0.94        40
                                failed_transfer       0.90      0.90      0.90        40
                           fiat_currency_support       0.92      0.82      0.87        40
                     get_disposable_virtual_card       0.92      0.88      0.90        40
                              get_physical_card       0.93      1.00      0.96        40
                            getting_spare_card        0.93      0.95      0.94        40
                           getting_virtual_card       0.84      0.95      0.89        40
                            lost_or_stolen_card       0.85      0.97      0.91        40
                           lost_or_stolen_phone       0.97      0.97      0.97        40
                            order_physical_card       0.97      0.90      0.94        40
                              passcode_forgotten       1.00      1.00      1.00        40
                          pending_card_payment       0.95      0.95      0.95        40
                       pending_cash_withdrawal       1.00      0.97      0.99        40
                                 pending_top_up       0.90      0.90      0.90        40
                              pending_transfer       0.86      0.78      0.82        40
                                   pin_blocked       0.97      0.88      0.92        40
                                receiving_money       0.95      0.95      0.95        40
                                 request_refund       1.00      0.93      0.96        40
                        reverted_card_payment?        0.86      0.95      0.90        40
                  supported_cards_and_currencies       0.86      0.95      0.90        40
                             terminate_account        0.98      1.00      0.99        40
                  top_up_by_bank_transfer_charge       0.97      0.88      0.92        40
                           top_up_by_card_charge       0.93      0.95      0.94        40
                        top_up_by_cash_or_cheque       0.95      0.93      0.94        40
                                  top_up_failed        0.84      0.93      0.88        40
                                  top_up_limits        0.93      1.00      0.96        40
                               top_up_reverted         0.87      0.85      0.86        40
                            topping_up_by_card         0.91      0.80      0.85        40
                     transaction_charged_twice         0.95      1.00      0.98        40
                          transfer_fee_charged         0.92      0.90      0.91        40
                         transfer_into_account         0.94      0.82      0.88        40
              transfer_not_received_by_recipient       0.87      0.85      0.86        40
                             transfer_timing          0.75      0.97      0.85        40
                     unable_to_verify_identity        0.93      0.95      0.94        40
                            verify_my_identity        0.86      0.93      0.89        40
                        verify_source_of_funds        0.98      1.00      0.99        40
                                  verify_top_up       1.00      1.00      1.00        40
                       virtual_card_not_working       0.97      0.93      0.95        40
                             visa_or_mastercard       1.00      0.93      0.96        40
                           why_verify_identity        0.95      0.88      0.91        40
                  wrong_amount_of_cash_received       0.97      0.93      0.95        40
              wrong_exchange_rate_for_cash_withdrawal       0.95      0.90      0.92        40

                                       accuracy                           0.92      3080
                                      macro avg       0.93      0.92      0.92      3080
                                   weighted avg       0.93      0.92      0.92      3080
```

```
cm = confusion_matrix(y_test, y_pred, labels=np.arange(num_classes))

plt.figure(figsize=(14, 12))
sns.heatmap(
    cm,
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)
plt.title("Test-set intent confusion matrix")
plt.xlabel("Predicted intent")
plt.ylabel("True intent")
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```




## Record the result for the comparison table

Accuracy, macro F1 and fine-tuning time are stored per dataset in `results.json` inside your output folder.

```
test_f1 = f1_score(y_test, y_pred, average="macro", zero_division=0)
print("Test macro F1:", round(float(test_f1), 4))

results_path = os.path.join(cfg.OUTPUT_DIR, "results.json")
results = {}
if os.path.exists(results_path):
    with open(results_path) as f:
        results = json.load(f)

results[cfg.DATASET] = {
    "model": cfg.PRETRAINED_NAME,
    "test_accuracy": round(float(test_accuracy), 4),
```

```
    "test_macro_f1": round(float(test_f1), 4),
    "epochs_run": len(finetune_history.history["loss"]),
    "finetune_seconds": round(float(finetune_seconds), 1),
}
with open(results_path, "w") as f:
    json.dump(results, f, indent=2)

pd.DataFrame(results).T
```

```
Test macro F1: 0.9241
```

| | model | test_accuracy | test_macro_f1 | epochs_run | finetune_seconds |
|---|---|---|---|---|---|
| **banking77** | distilbert/distilbert-base-uncased | 0.924 | 0.9241 | 5 | 507.1 |

## 13. Create a reusable class-based predictor

```
class IntentPredictor:
    def __init__(self, model, tokenizer, class_names, max_length=128):
        self.model = model
        self.tokenizer = tokenizer
        self.class_names = list(class_names)
        self.max_length = max_length

    def predict(self, text, top_k=3):
        encoded = self.tokenizer(
            text,
            max_length=self.max_length,
            truncation=True,
            padding="max_length",
            return_tensors="tf"
        )

        inputs = {
            "token_ids": encoded["input_ids"],
            "attention_mask": encoded["attention_mask"]
        }

        logits = self.model(inputs, training=False)
        probabilities = tf.nn.softmax(logits, axis=-1)[0]

        values, indices = tf.math.top_k(
            probabilities,
            k=min(top_k, len(self.class_names))
        )
```